How we judge "better": evaluation metrics

This document defines exactly how the real-LLM evaluation decides whether the **ContextAI MCP tools** help. It is the scoring contract — fixed *before* running so results can't be cherry-picked.

The experiment in one picture

For every one of the **48 queries** (12 across each of `flask`, `click`, `httpx`, `rich`), the **same** OpenAI model answers the **same** question **twice**:

Arm	Tools the model can call
A — Traditional	`grep`, `read_file`, `list_dir`
B — ContextAI + Traditional	the same tools **plus** ContextAI's MCP tools (`build_graph`, `find_node`, `get_context`, `get_edge_path`, …)

The model is **agentic**: it decides which tools to call and when. Everything except the available tool set is identical between arms (same prompt, same model, same query).

What we measure (three axes)

Axis 1 — Answer quality (LLM-as-judge)

A separate **judge model** grades answers. Two complementary methods:

- **Blind pairwise preference (headline).** The judge sees the question, a reference answer key (when available), and the two answers with **labels stripped and order randomized**. It picks *A wins / B wins / tie* with a reason. → **Headline number = Arm B win-rate over Arm A.**
- **Absolute rubric (0–3 each).** Each answer is scored independently on: | Criterion | Meaning | |---|---| | Correctness | no false statements about the code | |

Completeness | covers the real answer, not a fragment | | Groundedness | claims cite real files/symbols that actually exist |

→ per-answer quality = mean of the three (0–3).

Axis 2 — Objective correctness (no judge, no bias)

For the checkable query types — **callers, callees, impact** — we build a ground-truth **answer key** automatically (jedi + the code graph), then compare it to the set of symbols the model *states* as its answer:

```
precision = |model_answer ∩ key| / |model_answer|
recall    = |model_answer ∩ key| / |key|
F1        = 2·P·R / (P + R)
```

The model returns a small machine-readable list alongside its prose so this is scored exactly, not guessed.

Axis 3 — Efficiency (no judge)

Per query, per arm, we log:

Metric	Why it matters
tool calls	how many steps to the answer
tokens (in/out)	cost of reaching the answer
files opened	how much the agent had to read
latency (s)	wall-clock to answer

Hypothesis: Arm B reaches a correct answer in **fewer** steps/tokens.

Behavioral (descriptive)

In Arm B we also record how often the model *chose* the ContextAI tools vs grep — if it ignores them, that's a finding too.

Bias controls (so the result is trustworthy)

- Judge is **blind** to which arm produced an answer; answer order is **randomized**.
- Judge follows a **fixed rubric** and is given the objective answer key as reference.
- **Judge ≠ answerer** (a different model instance) to avoid self-preference.
- Judge agreement is **validated** on a few hand-scored samples; agreement is reported.
- Ties are reported honestly; both pairwise and absolute scores are shown.

Run protocol (the defaults)

- **Answerer:** the strongest available OpenAI reasoning model (confirmed at run time).
- **Judge:** a separate strong model instance, blind.
- **Repeats:** each (query, arm) is run **3×** and averaged to reduce variance.
- **Budget:** the agent loop is capped at 25 tool calls per query (safety bound).
- Total: 48 queries × 2 arms × 3 repeats = **288 agent runs**, then judged.
- The code graph for each repo is built once up front with the real `build_graph` tool.

What "better" looks like in the final report

Arm B "wins" if it shows: 1. a **pairwise win-rate > 50%** over Arm A (higher-quality answers), **and/or** 2. **higher objective F1** on callers/callees/impact, **and/or** 3. **lower effort** (fewer tool calls / tokens) for equal-or-better quality.

The strongest story — and the one we expect — is **Arm B is at least as accurate while using meaningfully fewer steps**, i.e. ContextAI makes the model both better and cheaper.

Worked scoring example

Query `httpx-02` (callers of `Client._send_single_request`): - Answer key (jedi): `{_client.py::_send_single_request is called by _send_handling_redirects}`. - Model (Arm B) states: `{_send_handling_redirects}` → precision 1.0, recall 1.0, F1 1.0. - Model (Arm A) states: `{_send_handling_redirects, send}` (over-broad guess) → precision 0.5, recall 1.0, F1 0.67. - Efficiency: Arm B used 2 tool calls (`find_node` + `get_context`); Arm A used 6 (`grep` + 5 reads). - Judge (blind) prefers the B answer for being precise and grounded.